

Exercise - AEM Content Discovery Agent

Time estimate	8 hours
---------------	---------

Format	Command-line tool - Python or Node.js
--------	---------------------------------------

Background

A large retail brand has 40,000+ content pieces stored in AEM Sites. Content authors spend hours doing work the system should handle: searching for reusable content, checking brand guidelines, and verifying whether similar pages already exist. By the time an author starts writing, they should already know what is available to reuse.

What the agent produces

The agent accepts a plain-text content brief and returns three things:

01 Top 3 matching content fragments

Fragments from the corpus ranked by relevance, each with a score and a one-line reason for the match.

02 Gap analysis

Topics the brief requires that no existing fragment adequately covers — what the author needs to write from scratch.

03 Draft outline

A suggested page structure that cites fragment IDs for reusable sections and clearly marks what needs to be written new.

Deliverables

Part 1	Synthetic content library Create the test data your agent will search
-----------	---

Build a set of 15–20 JSON content fragments representing a realistic AEM Sites corpus for a retail brand. These become the index your agent searches against. Each fragment must include:

- id — a unique fragment identifier (e.g. frag_001)
- title — the content page or component title
- category — content type (e.g. product-story, care-guide, seasonal-campaign)
- targetAudience — demographic description
- brandGuidelinesApplied — which brand rules apply (e.g. sustainability-voice, premium-tone)
- locale — e.g. en-gb, en-us, fr-fr
- lastModified — ISO date
- content — at least 100 words of realistic body text

**Part
2****Runnable agent script**

The command-line agent

Build the agent. It must run with a single command and produce all three outputs for any content brief passed to it. No web UI is required.

You may use any framework, any LLM provider, and any vector store that runs locally. Document your choices in Part 3.

**Part
3****Architecture document**

One page explaining your decisions

Write a short document / deck that justifies the significant decisions you made. Do not describe what you built. Explain why you built it that way.

At minimum, cover these four decisions:

1. Embedding model : why this model for this content and use case?
2. Chunking strategy : how did you split content, and why?
3. Retrieval method : vector search, keyword search, or hybrid? Why does it matter here?
4. Why an agentic approach? Which orchestration pattern and why?

Example input brief

Use this brief to test your agent. Your README must show the actual output your agent produces for this input.

I'm writing a landing page for our new sustainable winter collection.
Target audience is eco-conscious women aged 25-40 in the UK market.
The page needs to cover: our recycled materials sourcing story,
care instructions that extend garment life, and seasonal styling
tips. Tone should match our premium brand voice. The page will sit
under /en-gb/collections/winter-sustainable.

How to submit

Share a GitHub repository containing:

- All agent code, prompt logs
- Your content corpus JSON file
- A README with setup instructions and the agent's output for the brief above
- Your architecture document / deck (a separate Markdown file or section in the README or ppt)

If any requirement is unclear, state your assumption in the README and proceed.